PROOF OF CONCEPT

Same-Tail Delay Propagation in Australian Domestic Aviation

A first technical test of whether delay carries forward through an aircraft's rotation across a single day — using public ADS-B data via OpenSky.

Purpose of This Test

This is a small, deliberately narrow test — not the full project. The aim is to answer one question before building any visualisation pipeline:

> **Core question**
>
> Can a believable multi-leg rotation for a single aircraft be reconstructed from public ADS-B data, and is there at least one visible instance of delay carrying forward from one leg to the next?

If the answer is yes, the larger cascading-delay visualisation concept is worth building. If every leg looks fully recovered/buffered, that's also useful — it means the hypothesis needs adjusting (e.g. testing a known bad-weather day) before committing a semester to it.

Step 1 — Confirm OpenSky Historical Access

This has a waiting period, so do this first, separately from any coding.

1. Register a free account at opensky-network.org.
2. Apply for historical / Trino database access specifically — this is separate from the live API and needs a short statement of academic/research purpose. State QUT enrolment and the unit name.
3. Install the traffic Python library locally:

```
pip install traffic
```

Approval can take a few days. Start this in week 1, not when ready to code.

Step 2 — Pick a Test Aircraft and Day

Rather than guessing a tail number blind, query by airport and time window first, find a busy aircraft, then re-query by that aircraft across the full day.

Choose an ordinary, non-holiday weekday for the first test — this keeps the result easy to interpret before adding weather or seasonal noise.

Step 3 — Run the Proof-of-Concept Script

Part A — find candidate aircraft departing Sydney that day

```
from traffic.data import opensky
import pandas as pd

start = "2026-05-06 00:00:00"
stop  = "2026-05-06 23:59:59"

syd_departures = opensky.history(
    start=start,
    stop=stop,
    departure_airport="YSSY",
    selected_columns=[
        "time", "icao24", "callsign",
        "lat", "lon", "baroaltitude",
        "velocity", "vertrate"
    ]
)

print(syd_departures.groupby("icao24")["callsign"]
      .nunique().sort_values(ascending=False).head(10))
```

This produces a shortlist of aircraft (by icao24) appearing multiple times leaving Sydney that day — a sign of an aircraft doing several legs.

Part B — pull that aircraft's full day, any airport

```
candidate_icao24 = "7CXXXX"  # replace with one from Part A's output

full_day = opensky.history(
    start=start,
    stop=stop,
    icao24=candidate_icao24,
    selected_columns=[
        "time", "icao24", "callsign",
        "lat", "lon", "baroaltitude",
        "velocity", "vertrate"
    ]
)
```

```
full_day = full_day.sort_values("time")
print(full_day[["time", "callsign", "lat", "lon", "baroaltitude"]])
```

Step 4 — What to Look For

This is the actual test. Working through full_day, check for:

1. Distinct legs — look for the callsign changing, or altitude dropping to near-zero and climbing again repeatedly. Each cycle is a separate flight leg. Legs can be segmented programmatically by detecting when baroaltitude crosses below roughly 500 ft (a landing) and then climbs again (the next takeoff).
2. Turnaround time between legs — for each landing-to-next-takeoff pair, calculate the time gap and compare it to a typical turnaround for that route/aircraft type. A short run of "normal" days may need to be checked first to establish that baseline.
3. The actual hypothesis test — does a longer-than-typical turnaround or late departure on leg 1 correlate with a delayed departure on leg 2? With one aircraft on one day this will not be statistical proof. The question at this stage is simply whether the signal exists at all, or whether the airline appears to fully absorb delay via schedule buffer before the next leg.

What Success Looks Like at This Stage

> **Not the goal**
>
> Proving the full cascading-delay thesis with one aircraft.

> **The actual goal**
>
> Confirming that a believable multi-leg rotation can be reconstructed from ADS-B data, and that at least one visible instance of delay carrying from one leg to the next exists.

Two outcomes, both useful:

- Signal visible — proceed to scaling up across more aircraft, more airports, and the full visualisation pipeline with confidence.
- Every leg looks fully buffered/recovered — test a known bad-weather day next, since the effect may only surface under higher system stress, before redesigning the hypothesis.

Next Step (Not Yet Built)

Once real data comes back from this test, the next piece of work is leg-segmentation logic — turning the raw full_day dataframe into clean, automatically-labelled "leg 1, leg 2, leg 3…" rows, ready to feed into the wider delay-cascade visualisation.